Murine antibody humanization via frequentist analysis, simulated annealing, and support vector machine algorithm

Contents

1 Intro

The aim of this GitHub repository is to delve deeper into biophysics, using simulated annealing and science data to humanize murine antibodies.

We use a frequentist analysis of the given data in `learn_human.txt` and `learn_mouse.txt` in order to measure entropies and correlations.

With those outputs we intend to fine-tune the simulated annealing algorithm for the humanization.

1.1 Raw data files

The `learn_human.txt` and `learn_mouse.txt` data entries are chains of 298 aminoacids each. Every aminoacid is represented by a character in this string: `-ACDEFGHIKLMNPQRSTVWY`. The `-` is a gap in the chain, as used in the notation convention.

1.2 Aminoacid's properties

Every aminoacid is a certain chemical compound. Thus, it has chemical properties, which vary form aminoacid to aminoacid. The used properties in this project are (in order): porperty-gap, hydrophobic, aromatic, aliphatic, polar, small, minuscule, positively charged, and negatively charged. Every aminacid has asigned at least one property, so for consistency we added the property version of the gap: property-gap, denoted as `-(PROP)`.

2 Representation and modelization of data

2.1 Aminoacids and properties

Every aminoacid will be simulated/coded as a 21-dimensional vector of real numbers between 0 and 1 yuxtaposed to a 9-dimensional vector of real numbers between 1 and 0. That is, some aminoacid in our aminoacid space, $a \in \mathcal{A}$, will be $a = v \times w \in [0,1]^{21} \times [0,1]^9 = \mathcal{A} \subset \mathbb{R}^{21} \times \mathbb{R}^9$.

- The first list will be a 21-dimensional vector, whose i-th component corresponds to the frequency of the i-th aminoacid from the aforementioned aminoacid string.
- The second list will be a 9-dimensional vector, whose i-th component corresponds to the frequency of the i-th property from the aforementioned property list.

As you can see, these variables are context-dependent, since there is no reference for the frequency of aminoacids or properties. Therefore, if it is one that represents an actual aminoacid, all frequencies may be 0 but one of them, which should be 1. Same will happen for the property vector. We will call this a *concrete aminoacid*.

Example: Let's encode the aminoacid cysteine (`C`). It is the third in the aminoacid list, thus, the first vector should be $v = (0, 0, 1, 0, ..., 0)$. Cysteine is hydrophobic and small, then: $w = (0, 1, 0, 0, 0, 1, 0, 0, 0)$, since hydrophobic is in the 2nd position and small in the 6th.

Now changing the context, another usual setting is the so called *Schrödinger's aminoacid*. That will be the average aminoacid, whose elements in both vectors will be *separately* the frequencies of every appearing aminoacid.

Separately means that the sum of the components of the first list is 1 and the sum of the components of the second list will also be 1.[1] More specifically: $\sum_{i=1}^{21} v^i = 1 \ \wedge \ \sum_{i=1}^{9} w^i = 1$.[2]

[1] This is true for every kind of aminoacid, whether it is concrete or Schrödinger's.
[2] Using superindexes for convenience.

2.2 Chains

A chain is an array of 298 aminoacids and it represents a whole antibody. Just as before, we will have concrete (all components must be concrete aminoacids) and Schrödinger's chains (the rest).

This is a $298 \times (21 + 9)$ matrix of split row conditioned frequencies. We will name the space of all possible chains the chain space, $\mathcal{C} = \mathcal{A}^{298}$. Thus, $c \in \mathcal{C}$ will have the form $c = (c_1, ..., c_j, ..., c_{298})$, where $c_j = v_j \times w_j$ and $v_j = (v_j^1, ..., v_j^i, ..., v_j^{21}),\ \forall j \in [1, 298] \subset \mathbb{N}$.

3 Expressions and Formulas

3.1 Entropies

We quantify the variability of the aminoacids in a Schrödinger's chain with entropies, applying its expression to both vectors of every aminoacid in that chain. This means the result will be a 298 element list of pairs of entropies.

The compute them as follows:

$$S_{v,j} = -\sum_{i=1}^{21} v_j^i \log_2 v_j^i, \quad S_{w,j} = -\sum_{i=1}^{9} w_j^i \log_2 w_j^i, \quad S_j = \big(S_{v,j}, S_{w,j}\big) \quad \forall j \in [1, 298] \subset \mathbb{N}$$

That previous one is the Shannon entropy, but we may need to use some others:

- Linear entropy: $s = \sum_i p_i(1 - p_i)$
- Rényi entropy: $s_\alpha = \frac{1}{1-\alpha} \sum_i p_i^\alpha,\ \alpha \in [0, 1)$
- Tsallis entropy: $s_\alpha = \frac{1}{q-1}\big(1 - \sum_i p_i^\alpha\big),\ \alpha \in [0, 1)$

3.2 Correlation

The measure of correlation between positions in a chain used in this project is mutual information. It is an entropy measure of how much information (in bits) we can deduce of one random variable given another one, and vice versa.

Mutual information is given by:

$$I(X;Y) = \sum_{x \in \mathcal{X}} \sum_{y \in \mathcal{Y}} P\big(\{X = x\} \cap \{Y = y\}\big) \log_2 \frac{P\big(\{X = x\} \cap \{Y = y\}\big)}{P\big(\{X = x\}\big) P\big(\{Y = y\}\big)},$$

where X and Y are discrete random variables that take values, x and y, in $\mathcal{X}$ and $\mathcal{Y}$. Since we can identify our vectors, v and w, with distribution functions, we are able to apply this method.

Note that $I(X;Y) = I(Y;X)$, since the product and intersection are symmetric.

Also,

$$I(X;X) = \sum_{x \in \mathcal{X}} P\big(\{X = x\}\big) \log_2 \frac{P\big(\{X = x\}\big)}{\big(P\big(\{X = x\}\big)\big)^2} = -\sum_{x \in \mathcal{X}} P\big(\{X = x\}\big) \log_2 P\big(\{X = x\}\big) = S(X),$$

where $S(X)$ is the entropy of the random variable X.

In practice, we will get a 298×298 symmetric matrix for aminoacids (v-part of the chain) whose diagonal is the entropies of every position. However, for our correlation indeces we would rather have all 1's in the diagonal and the other elements going from 0 to 1. We normalize it as follows:

$$i(X;Y) = \frac{I(X;Y)}{\min\{S(X), S(Y)\}}.$$

When $\min\{H(X), H(Y)\} = 0$, the position is fully conserved and $I(X;Y) = 0$ as well. In this case, the code outputs $i(X;Y) \equiv 0$, by convention.

4 Metropolis

Important information: Metropolis assumes CDRs are located in the same position for all antibodies being analyzed; in order for this to be valid, sequences need to be properly aligned beforehand.

4.1 Simulation Workflow: From Batch to Single Run

Two functions are used for running the simulation: a high-level manager (`mega_metropolis`), and a core simulation engine (`run_metropolis`). This design allows for parallelized execution of simulations on multiple initial murine sequences.

4.1.1 The `mega_metropolis` Manager

`mega_metropolis`'s primary responsibilities are:

1. **Data Loading:** It begins by reading a list of initial murine sequences (which we call seeds) from a specified file and saves them for later use.
2. **Batch Management:** It creates a timestamped main directory for the entire simulation batch (format `results/metropolis/YYYY-MM-DD_HH_MM_SS_.../`). .
3. **Parallel Execution:** It iterates through each seed sequence using an OpenMP parallel loop. For each seed, it creates a dedicated subdirectory (e.g., `seq_1`, `seq_2`).
4. **Replication:** For each seed, it calls the `run_metropolis` function `n_metropolis` times. This allows for multiple independent simulation runs starting from the same seed, providing statistical significance to the results. The output of each run is saved to a separate file (e.g., `run_1.txt`, `run_2.txt`) within the corresponding seed's subdirectory.

4.1.2 The `run_metropolis` Engine

The `run_metropolis` function performs a single, complete simulated annealing run on one given sequence:

1. **Initialization:** It takes an initial sequence, the human Schrödinger chain, and the pre-calculated matrix of β values.
2. **Simulated Annealing Loop:** It iterates through each row of the β matrix, representing a step in the "cooling" schedule.
3. **Metropolis Sweeps:** For each β^i step, it performs a specified number of Metropolis sweeps (`n_sweeps`). A single sweep involves attempting a mutation at every position in the sequence.
4. **State Acceptance:** We begin by proposing a new state, selecting a new AA (including the gap character) uniformly from the 21 possibilities (ensuring it's different from the current one). Each proposed mutation is accepted or rejected based on the change in total energy (ΔE) and the current, position-specific, β_j^i value, following the Metropolis criterion: accept if $\Delta E < 0$ or if a random number is less than $e^{-\beta_j^i \Delta E}$.
5. **Data Logging:** After the simulation is complete, it calls a dedicated function to write a detailed report file containing the initial parameters, the final sequence and energy for each β step, and overall acceptance statistics.

4.2 Simulated Annealing Schedule (β)

The β-values, which are inverse of the temperature ($1/k_BT$), are not constant; instead, they are defined through a mechanism, which we call the "engine":

- **Engine:**

$$\beta_i^j = k_i \cdot \frac{\alpha}{S_j + \varepsilon}, \quad k_i = (c_r)^i \ \ \forall i \in [1, N_\beta], j \in [1, L_{\text{chain}}]$$

where c_r is the cooling rate, ε avoids division by 0, α is a scale factor, and S_j is entropy for position j in human reference Schrödinger chain.

4.3 Energy Function

The total energy, E_{total}, is a weighted sum of three components, which we aim to minimize:

$$E_{\text{total}} = w_{\log} E_{\log} + w_{\text{prop}} E_{\text{prop}} + w_{\text{penalty}} E_{\text{penalty}}$$

The weights for each component (`WEIGHT_LOG`, `WEIGHT_PROP`, `WEIGHT_PENALTY`) are defined in 'head.h'.

1. **Log Humanness Energy, $\mathbf{E}_{\log}$**: This term quantifies how "human-like" a sequence is based on AA frequencies from a reference set of human sequences (the Schrödinger chain):

$$E_{\log} = -\sum_{i=1}^{L_{\text{chain}}} \log(p_i(a_i))$$

where $p_i(a_i)$ is the frequency of the amino acid a_i at position i in the human reference. A large penalty (`ZERO_FREQ_PENALTY_LOG`) is added for amino acids with a frequency below a small threshold (`EPSILON`).

2. **Property Distance Energy, $\mathbf{E}_{\text{prop}}$**: This term measures the similarity between the properties of the sequence and the human reference; it's calculated as the squared Euclidean distance in the property space for each position:

$$E_{\text{prop}} = \sum_{i=1}^{L_{\text{chain}}} \sum_{j=1}^{N_{\text{props}}} \left(\text{prop}_j(a_i) - \overline{\text{prop}_j(i)}\right)^2$$

where $\text{prop}_j(a_i)$ is the value of property j for amino acid a_i, and $\overline{\text{prop}_j(i)}$ is the average value of property j at position i in the Schrödinger chain.

3. **Wandering Penatly Energy, $\mathbf{E}_{\text{penalty}}$**: This term discourages the sequence from deviating too far from the original murine sequence; we thus only incentivize mutation if there's a relatively big gain in humanness. It consists of a penalty based on property distance and a (for now) constant penalty for any mutation:

$$E_{\text{penalty}} = \sum_{i=1}^{L_{\text{chain}}} \left[\left(\sum_{j=1}^{N_{\text{props}}} (\text{prop}_j(a_i) - \text{prop}_j(a_i^{\text{original}}))^2 \right) + P \cdot \delta_{a_i}^{a_i^{\text{original}}} \right]$$

where a_i^{original} is the amino acid at position i in the original murine sequence, P is the mutation penalty (`AA_MUTATION_PENALTY`), and δ is the "generalized Kronecker delta" (logical if gate). This approach ensures each mutation has a minimum energy cost, while heavily penalizing big property changes.

4.4 Model Parameters

The following table summarizes the key parameters for metropolis in our simulations. These values are defined in `main.c` and `head.h`.

Parameter	Value[3]	Description
`n_betas`	50	Number of cooling steps in the annealing schedule.
`n_sweeps`	800	Number of Metropolis sweeps per β step.
`n_metropolis`	20	Number of independent runs per murine seed sequence.
`w_log`	0.4 (`head.h`)	Weight for the log-humanness energy component.
`w_prop`	0.4 (`head.h`)	Weight for the property-distance energy component.
`w_penalty`	0.2 (`head.h`)	Weight for the wandering penalty energy component.
`cooling_rate` (c_r)	1.02	Multiplicative factor for increasing k_i at each step.
`scale_factor` (α)	1.0	Global scaling factor for all β values.
`AA_MUTATION_PENALTY` (P)	1.5 (`head.h`)	Constant penalty added for any AA mutation away from the original.

[3] Provisional values.

5 Support Vector Machine

We intend to back up the humanness of the chain outputted by the simulated annealing algorithm with this binary-output method.

The main point of this method is to transform our data space into some other one. In such a way that we can separate the data points with a simple plane, a linear classifier.

We will represent each point with a n-dimensional vector, where n is the number of data points. The function that gives us those coordinates is called the kernel.

5.1 The kernel

Our kernel, $\mathrm{k} : \mathcal{C} \times \mathcal{C} \to \mathbb{R}$, will be the Gaussian radial basis function:

$$\mathrm{k}(\mathbf{x}_i, \mathbf{x}_j) = \exp\left(-\gamma \left\|\mathbf{x}_i - \mathbf{x}_j\right\|^2\right), \ \gamma > 0,$$

where $\{\mathbf{x}_i\}$ are the data points.

After applying the kernel we have a matrix $M \in \mathbb{R}^{n\times n}$, the Gram matrix, with the coordinates of our data points in the new space. That will be: $[M]_{i,j} = \mathrm{k}(\mathbf{x}_i, \mathbf{x}_j)$.

We also have abailable some other alternative kernel functions:

- Sigmoid function:

$$\mathrm{k}(\mathbf{x}_i, \mathbf{x}_j) = \tanh\Big[\gamma\big(\langle \mathbf{x}_i | \mathbf{x}_j \rangle + r\big)\Big]$$

- Polynomial inhomogenuous function:

$$\mathrm{k}(\mathbf{x}_i, \mathbf{x}_j) = \big(\langle \mathbf{x}_i | \mathbf{x}_j \rangle + r\big)^d$$

Being $d > 0$ and r real constants.

The Gram matrix and the kernel are the tools we need to craft the decision function.

5.2 Decision function

Using the lagrange multipliers method for minimization[4], we get the decision function, $f : \mathcal{C} \to \mathbb{R}$:

$$f(\mathbf{x}) = b + \sum_{i=1}^{n} \lambda_i s_i \, \mathrm{k}(\mathbf{x}_i, \mathbf{x}),$$

where λ_i are the lagrange multipliers obtained through said method, s_i is the *tag* of the i-th data point (may be $+1$, when human, or -1, when murine) and b is the bias, obtained as follows:

$$b = s_p - \sum_{i=1}^{n} \lambda_i s_i \, \mathrm{k}(\mathbf{x}_i, \mathbf{x}_p),$$

where the p-th data point is a support one, that is, the closest to the plane (or any of them) and such that $\lambda_p > 0$.

[4] See apendices for full explaination.

5.3 SVM results

Using the `learn_human`, `learn_mouse`, `test_human`, and `test_mouse` chain sets for learning and testing, we get outstandingly good results. In the next histograms, we have the distributions of the output of the decision function for `test_human` and `test_mouse` respectively:

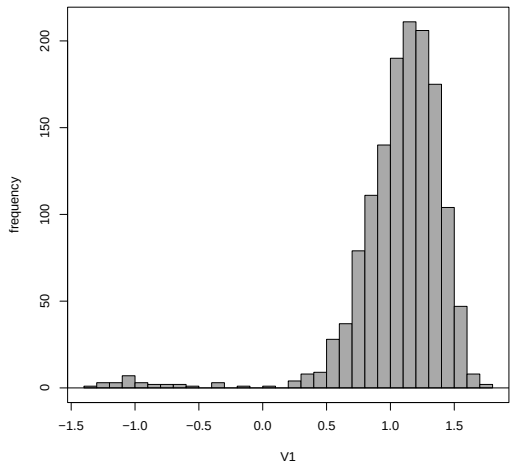

Figure 1: Count with respect to value of the output for the `test_human` chains.

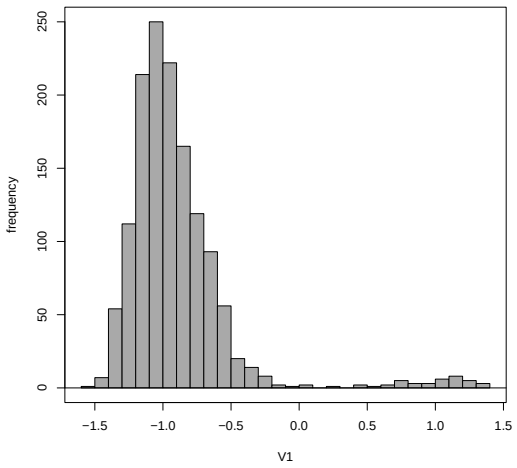

Figure 2: Count with respect to value of the output for the `test_mouse` chains.

This gets us to the confusion matrix:

	Human	Murine
Tagged as human	1360	41
Tagged as murine	28	1338

Counting human as positive, this gets us to a F1[5] of ~ 0.9753[6]. On the other hand, being murine the positives, we have an F1 of ~ 0.9749[7]. This takes us to an overall accuracy[8] of ~ 0.9751[9].

However, those tests were taken with a completely heuristic upper limit, we will call λ_t, for the λ vector. Since we cannot determine annalytically λ_t, we must adjust it as a hyperparameter with a validation data set.

[5] $\mathrm{F1} = \frac{2 \cdot \mathrm{TP}}{2 \cdot \mathrm{TP} + \mathrm{FN} + \mathrm{FP}}$
[6] Precisely: $\mathrm{F1_H} = 0.975259949802797$.
[7] Precisely: $\mathrm{F1_M} = 0.974863387978142$.
[8] $\mathrm{Acc} = \frac{\mathrm{TP} + \mathrm{TN}}{\mathrm{TP} + \mathrm{TN} + \mathrm{FP} + \mathrm{FN}}$
[9] Precisely: $\mathrm{Acc} = 0.975063245392121$.

5.3.1 Validation

We have merged all data sets and split them into `learn`, `validation`, and `test`, with 70%, 15%, and 15% of the chains respectively.

Through this method we get an optimal value for $\lambda_t = 10.0$, having tried with powers of 10 from 0.01 to 1000. Using that hyperparameter, we get the following confusion matrix

	Human	Murine
Tagged as human	404	4
Tagged as murine	6	254

And F1 values and accuracy:

$$\mathrm{F1_H} = 0.9878^{[10]} \qquad \mathrm{F1_M} = 0.9788^{[11]} \qquad \mathrm{Acc} = 0.9836^{[12]}$$

The following histogram shows the distribution of the sign-corrected certainty degree, that is, the output of the decision function but sign inverted when it ought to be negative. This way the positive numbers are correct ones and negative are failed tagging:

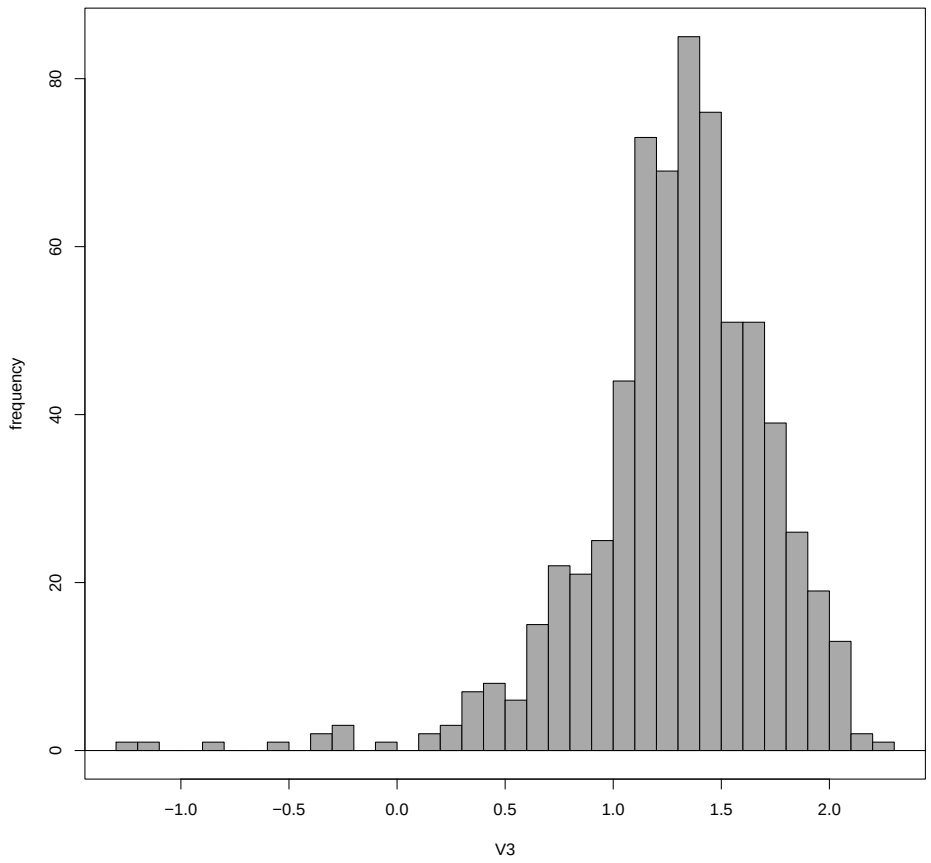

Figure 3: Count with respect to sign-corrected certainty degree using self-split learn, validation and test datasets.

[10] Precisely: $\mathrm{F1_H} = 0.987775061124694$.
[11] Precisely: $\mathrm{F1_M} = 0.978805394990366$.
[12] Precisely: $\mathrm{Acc} = 0.98355754857997$

Appendix

Support Vector Machine formula deduction

We have a cloud of points in a higher-dimensional space and we want a plane that separates the ones marked human and the one marked murine[13]. But we do not only want *one* that does that, we need the plane that does that the best: that is the one that maximizes the distance to the closest points.

We *must* assume that the two groups of points are separable by a plane in our space and let us say the distance[14] between the two clouds is $2d$; we will need this later.

Our solution will have the form:

$$\boldsymbol{w}^T \boldsymbol{X} + b = 0,$$

where $\boldsymbol{w}$ is the vector perpendicular to the solution plane, $\boldsymbol{X}$ is a variable-coordinate vector and b is a real number we will call bias.

Since the norm of $\boldsymbol{w}$ is arbitrary, we can set $\|\boldsymbol{w}\| = 1/d$. Thus, any of our data points inputted in the funtion $\boldsymbol{x} \mapsto \left|\boldsymbol{w}^T \boldsymbol{x} + b\right|$ will give an output greater or equal to 1 (the nearest ones will give 1, otherwise strictly grater than 1).[15]

Furthermore, we know whether the result will be negative and then inverted by the absolute value. The ones such that $\boldsymbol{w}^T \boldsymbol{x}_i + b \leq -1$ will have also the negative tag: $s_i = -1$. Thus,

$$\left(\boldsymbol{w}^T \boldsymbol{x}_i + b\right) s_i \geq 1 \ \forall i \in \{1, \ldots, n\}.$$

Maximizing d is equivalent to minimizing $\|\boldsymbol{w}\|$, which is again equivalent to minimizing $\frac{1}{2}\|\boldsymbol{w}\|^2$, this will come handy after some computations.

We have a optimization problem subject to restrictions:

$$\underset{\boldsymbol{w},\ b}{\text{minimize}}\ \frac{1}{2}\|\boldsymbol{w}\|^2, \qquad \text{subject to } \left(\boldsymbol{w}^T \boldsymbol{x}_i + b\right) s_i \geq 1 \ \forall i \in \{1, \ldots, n\}$$

We define the Lagrangian with the classification margin:

$$\mathcal{L}(\boldsymbol{w}, b, \lambda) = \frac{1}{2}\|\boldsymbol{w}\|^2 - \sum_{i=1}^{n} \lambda_i (s_i(\boldsymbol{w}^T \boldsymbol{x}_i + b) - 1)$$

We set the partial derivatives to 0:

$$\nabla_{\boldsymbol{w}} \mathcal{L} = 0 \implies \boldsymbol{w} = \sum_{i=1}^{n} \lambda_i s_i \boldsymbol{x}_i \qquad \frac{\partial \mathcal{L}}{\partial b} = 0 \implies \sum_{i=1}^{n} \lambda_i s_i = 0$$

We substitute $\boldsymbol{w}$ back into the Lagrangian and apply the restriction for b:

$$\mathcal{L}(\lambda_1, \ldots, \lambda_n) = \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \lambda_i \lambda_j s_i s_j \boldsymbol{x}_i^T \boldsymbol{x}_j - \sum_{i=1}^{n}\sum_{j=1}^{n} \lambda_i \lambda_j s_i s_j \boldsymbol{x}_i^T \boldsymbol{x}_j - b\sum_{i=1}^{n} \lambda_i s_i + \sum_{i=1}^{n} \lambda_i$$

We simplify to obtain the dual formulation:

$$\mathcal{L}(\lambda_1, \ldots, \lambda_n) = \sum_{i=1}^{n} \lambda_i - \frac{1}{2}\sum_{i=1}^{n}\sum_{j=1}^{n} \lambda_i \lambda_j s_i s_j \boldsymbol{x}_i^T \boldsymbol{x}_j,$$

[13] Those are what we call tags: $+1$ for human and -1 for murine chains.

[14] Distance from one set to another is defined as the minimum distance from one point of one set to a point of the other one. Those two (or more) points will be called support vectors.

[15] Notice that $\boldsymbol{x}$ and $\mathbf{x}$ are different, in fact, $\boldsymbol{x}_i$ is the data point $\mathbf{x}_i$ in the transformed space ($\varphi(\mathbf{x}_i) = \boldsymbol{x}_i$, being φ the transformation of our space such that $\varphi(\mathbf{x}_i) \cdot \varphi(\mathbf{x}_j) = \boldsymbol{x}_i^T \boldsymbol{x}_j = \mathrm{k}(\mathbf{x}_i, \mathbf{x}_j)$).

We use our kernel notation for the inner products:

$$\mathcal{L}(\lambda_1, \ldots, \lambda_n) = \sum_{i=1}^{n} \lambda_i - \frac{1}{2} \sum_{i=1}^{n} \sum_{j=1}^{n} \lambda_i \lambda_j s_i s_j \, \mathrm{k}(\mathbf{x}_i, \mathbf{x}_j),$$

$$\text{with } \sum_{i=1}^{n} \lambda_i s_i = 0 \text{ and } \lambda_i \geq 0 \; \forall i \in \{1, \ldots, n\}$$

This way we find the lagrange multipliers, maximizing the Lagrangian subject to the restrictions given.

For our decision function we just want to know which side of the plane is on. We achieve that plugging the new data point into our plane equation:

$$\boldsymbol{w}^T \boldsymbol{x} + b = \left(\sum_{i=1}^{n} \lambda_i s_i \boldsymbol{x}_i^T \right) \boldsymbol{x} + b = \sum_{i=1}^{n} \lambda_i s_i \boldsymbol{x}_i^T \boldsymbol{x} + b = \sum_{i=1}^{n} \lambda_i s_i \, \mathrm{k}(\mathbf{x}_i, \mathbf{x}) + b$$

$$f : \mathbf{x} \mapsto b + \sum_{i=1}^{n} \lambda_i s_i \, \mathrm{k}(\mathbf{x}_i, \mathbf{x})$$

Also, since $(\boldsymbol{w}^T \boldsymbol{x}_p + b) s_p = 1$ for some support vector $\mathbf{x}_p$, expanding $\boldsymbol{w}$:

$$b = s_p - \sum_{i=1}^{n} \lambda_i s_i \, \mathrm{k}(\mathbf{x}_i, \mathbf{x}_p)$$